Vietnam National University Ho Chi Minh City
Ho Chi Minh City University of Technology
Faculty of Computer Science and Engineering

THESIS PROPOSAL – COURSE: CO5109

Cutting the Cost of LLM-Based Program Repair with Evidence-Driven Model Escalation



Faculty: Computer Science and Engineering
Semester: 261
Instructor: Dr. Le Xuan Bach, CSE-HCMUT
Student: Nguyen Thai Thanh Dat
Student ID: 2570387

Ho Chi Minh City, November 2026

Abstract

In LLM-based program repair, each round has two costs: the model call that proposes a patch and the test run that checks it. Known techniques make the test run cheap: the patched file can be compiled in process, the test runtime kept alive, and the tests that cannot reach the change skipped. What remains is the model call. A strong model repairs more bugs but costs about twenty times more per token than a cheap one, and a repair bot must pick a model before it knows how hard the bug is. Starting at the strong model pays its price for easy bugs. Staying at the cheap model burns its budget on bugs it cannot repair. And at a switch between models, what the first model learned is either dropped or carried in a form that may bias the second.

This proposal builds a repair bot that tries the cheap model first and *steps up* to the strong model only when the tests show that the session has stalled. A *Warm Substrate* pays the build and the start-up of the test runtime once per session instead of once per candidate, so the cheap model's extra candidates cost little to check. An *Evidence Memory* keeps every verdict, failing test and failure signature, and screens the candidates of both models. An *Escalation Controller* applies a rule, Step, that moves the session up a tier after a verified stall and writes a *handover note* from the evidence. A *Budget Guard* stops sessions that are stuck or out of budget and reports why. With a cold handover and an unchanged budget for the strong model, the bot repairs every bug that the strong model alone repairs on the same recorded sessions; what it changes is the cost.

The evaluation uses 100 Defects4J bugs. Sessions of each model alone are recorded once. Every escalation policy is then replayed on the same candidates and compared with a router that knows the outcome in hindsight. Live runs then measure what the handover note is worth to the strong model and, on bugs held out from that choice, whether the whole system lowers the cost per repaired bug. The saving is expected mainly in money and tokens; in time it depends on how long the cheap tier delays the bugs it cannot repair.

Keywords: automated program repair, large language models, model escalation, patch validation, cost analysis

Contents

Abstract **ii**

1 Introduction **1**
1.1 Problem Statement 1
1.2 Research Questions and Contributions 3
1.3 Scope 4
1.4 Structure 5

2 Background **6**
2.1 APR Pipeline 6
2.2 Cost Model of a Tiered Session 6
2.3 Escalation and Its Break-Even 7
2.4 Verified Progress and Stall 8
2.5 What Can Be Handed Over 9
2.6 Benchmark 9

3 Related Works **10**
3.1 LLM-Based Repair Systems 10
3.2 Validation, Repeats and Stopping in Current Tools 11
3.3 Cost Measurement in LLM-Based Repair 11
3.4 Classification of Cost-Reduction Techniques 12
3.5 Model Selection: Routing and Cascades 12
3.6 Other Cost-Reduction Techniques 14
3.7 Research Gap 15

4 Proposed System **16**
4.1 Usage Scenario and Requirements 16
4.2 System Overview 17
4.3 Module M0: Warm Validation Substrate 18
4.4 Module M1: Evidence Memory 20
4.5 Module M2: Escalation Controller 21
4.6 Module M3: Budget Guard 23
4.7 When the System Helps 24

5 Evaluation Plan **25**
5.1 Bugs and Setup 25
5.2 What Is Compared 26
5.3 How Cost Is Measured 27

5.4 E0 and E1: Single-Tier Sessions, Audited 28
5.5 E2: Escalation Policies by Replay 29
5.6 E3 and E4: Handover and Whole System, Live 30
5.7 Metrics 30
5.8 Statistics 30
5.9 Compute and API Budget 31
5.10 Pre-Registration 32

6 Expected Outcomes **34**

7 Work Plan and Conclusion **36**
7.1 Work Plan 36
7.2 Risks 36
7.3 Threats to Validity 37
7.4 Future Work 38
7.5 Conclusion 38

References **39**

List of Figures

1.1 One session of a bot with two models. 2

4.1 One repair session with the tiered bot. 17
4.2 An example handover note (illustrative values). 22
4.3 An example give-up report (illustrative values). 24

5.1 The first prompt for Defects4J bug Lang-65. 29

List of Tables

1.1 Scope of the study. 4

2.1 What each family of techniques can save at most. 7

3.1 Representative LLM-based repair systems, what they optimise, and how they use validation. 10
3.2 The seven groups of cost reduction. 13
3.3 Model selection, by the signal the judge reads. 13

4.1 What a repair bot in CI needs, and the evidence. 16
4.2 The four modules. 18
4.3 One session of a bug in `DateUtils` (illustrative). 18
4.4 What one candidate pays with and without M0. 20

5.1 The five experiments. 25
5.2 The policies compared. 26
5.3 Feedback to the strong tier after each round. 28
5.4 Metrics. 31
5.5 Calls and audits, as upper bounds. 32
5.6 Provisional values and the rules that fix them. 33

6.1 Possible outcomes and how they will be read. 34

7.1 Work plan. 36
7.2 Main risks and how they are handled. 37

Chapter 1

Introduction

1.1 Problem Statement

Context. Automated program repair (APR) fixes a faulty program without a human writing the patch [1]. Most tools work in a loop: they generate a candidate patch and run the test suite on it; a candidate that passes every test is called *plausible* [2]. Today the candidate comes from a large language model (LLM). If a test fails, the tool shows the failure to the model and asks again [3]. Each round of this loop has two costs: the model call and the test run.

Need. The cost per bug is now reported next to the repair rate: ChatRepair spends about \$0.42 per bug [3] and RepairAgent about 270,000 tokens per bug [4]. Long, costly sessions are named as an obstacle to using LLM-based repair in practice [5]. The practical need is to *lower the cost per repaired bug without repairing fewer bugs.* A typical setting is a repair bot in continuous integration (CI), which is where developers most want repair to run [6]. Such a bot runs on every failing build, so it must also choose a model. Prices differ widely within one model family: GPT-6 Sol is listed at \$2 and \$10 per million input and output tokens, and GPT-6 Luna at \$0.10 and \$0.50, a factor of twenty [7].

What is already known. Three observations from the literature set the direction.

1. **Validation can be made cheap.** Much of what a plain validation pays is overhead: the candidate is rebuilt and the test runtime started again, although the project and the runtime do not change during a session. On-the-fly validation removes both and speeds classical repair up by one to two orders of magnitude [8, 9]. Test selection and prioritisation cut what is left [10–12]. None of these techniques depends on where a patch comes from, so this thesis adopts them as they are.
2. **A model repeats itself, and a small model gains little from being told.** 65 % of the patches of ChatRepair were duplicates of earlier ones [13]. Shown its failed attempt, a model of up to 7B parameters reproduces a near-identical program in 33 % to 68 % of retries, against 2 % to 14 % when it is simply asked again [14]. Stronger models do use feedback, and conversational repair is built on it [3], although what it gains depends on the model that reads it [15].
3. **What crosses a model switch can help or hurt.** A router for software agents restarts the strong model from scratch, on the stated ground that the weak model's reasoning biases it [16], and a weak draft can lower the result of

the model that revises it [17]. Yet an agent restarted with its interrupted diff resolved more issues than one restarted cold [18].

Read together: validation can be made cheap, the outcome of a test is the one part of a failed attempt that is verified, and the model that produces the evidence is not always the model that can use it.

Problem. The cost that remains is on the generation side. A bot with two models can waste it in three ways (Figure 1.1):

- **Strong price for easy bugs.** A bot that starts at the strong model pays its price on bugs the cheap model would have repaired. Low-cost models repair over half of moderately complex bugs [19], and smaller models repair some bugs that larger ones miss [20].
- **Cheap budget on hopeless bugs.** A bot that stays at the cheap model spends its whole budget on the bugs it cannot repair, and failed sessions are the expensive ones [21].
- **Evidence lost, or carried blindly, at the switch.** When a session moves to another model, what the first model learned is either dropped or passed on. Dropping it wastes verified facts. Passing it on can bias the next model towards the first model's mistakes [16, 17].

How large each waste is in test-driven repair is unknown. Studies of model routing and early stopping target issue-resolution agents, decide with learned predictors and count tokens only (Chapter 3). None uses the tests the bot already runs as its signal, prices the validation of the extra candidates, or measures what verified evidence is worth to the next model.

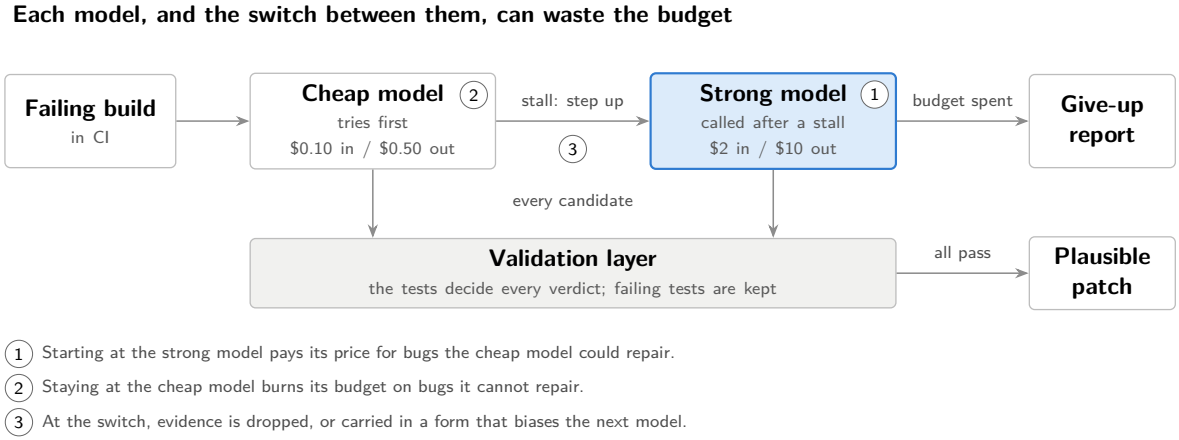


Figure 1.1: One session of a bot with two models; prices are per million tokens [7]. The marks are the three wastes: (1) the strong price paid on a bug the cheap model could repair; (2) the cheap budget spent on a bug it cannot repair; (3) evidence dropped at the switch, or carried in a form that biases the next model.

Goal. This thesis builds a *tiered repair bot*: it tries the cheap model first and steps up to the strong model only when the tests show a stall. It has four modules:

1. **Module M0, Warm Substrate**, makes one validation cheap. It compiles the patched file in process, loads it into a test runtime that stays alive, and runs the tests in a fixed schedule, so the build and the start-up are paid once per session instead of once per candidate. Cheap candidates then cost little to check.
2. **Module M1, Evidence Memory**, keeps what the tests have shown: the verdict of every candidate, the tests that failed and how they failed. It answers repeats without a test run and screens the candidates of both models.
3. **Module M2, Escalation Controller**, decides which model proposes next. Its rule, Step, moves the session up a tier after a verified stall and writes a handover note from the evidence.
4. **Module M3, Budget Guard**, keeps the cost per bug within a budget and reports why a session gave up.

M0 and M1 are the bot's *validation layer*: they never change whether a patch is accepted. M2 changes which model is asked, so its effect on the repair rate is measured; with a cold handover it cannot lower it (Section 2.3). M3 can end a session that would still have found a patch, so its trade-off is measured on its own.

Why this direction. The choice follows from the size of each lever. The validation side has mature techniques with measured savings [8–10], and none of them touches the model call. The price gap between tiers is a factor of twenty, and it applies to the part of the bill those techniques leave. Studies in neighbouring settings report savings of this order: 26 % on average for a cascade in code completion [22], up to 40 % for strong-weak collaboration in issue resolution [23], and 32 % on average for early termination [24]. Whether such savings hold in a test-driven loop, where the tests themselves can be the signal, has not been measured.

1.2 Research Questions and Contributions

- **RQ1: Where does the cost of a repair session go, for each model?** The cost per repaired bug, split into the model calls of each tier, building the candidate, starting the test runtime and executing tests, and how much of it falls on bugs the session never repairs. The split bounds what escalation can save.
- **RQ2: How cheap does the validation layer make one candidate?** The Warm Substrate and its test schedule measured against the per-candidate build and restart they replace, with any disagreement in verdict. This fixes the price of a cheap-tier attempt.
- **RQ3: Does cheap-first escalation lower the cost per repaired bug, and when should a session step up?** Step is compared with each model alone, with a switch after a fixed number of candidates, and with a router that knows the outcome in hindsight.
- **RQ4: What is the cheap tier's evidence worth to the strong tier?** A cold start is compared with a handover note of verified evidence and with the

full transcript: rounds to repair, tokens, and how often the strong model repeats what was already refuted.

- **RQ5: Does the whole system lower the cost per repaired bug?** Against a standard conversational loop with the strong model, on bugs that played no part in choosing its settings, without a lower repair rate.

Contributions.

C1 **A cost breakdown of LLM-based repair by model tier and validation step**, with the most that escalation and each other kind of technique could save. No published study reports this split for a test-driven loop.

C2 **Step, an escalation rule driven by verified evidence**, evaluated by replay on recorded sessions and against a hindsight bound. The bound says how much any router, learned ones included, could add in this setting.

C3 **A measurement of evidence handover across models**: whether verified test evidence helps the next model where drafts and reasoning are reported to bias it [16, 17].

C4 **A tiered repair bot** of four modules on a warm validation substrate, released with its data.

C1, C2 and C3 are the main research contributions. The Warm Substrate and the Budget Guard are supporting modules: they claim no new mechanism, and they are there because escalation depends on cheap validation and on a bounded budget.

1.3 Scope

Table 1.1: Scope of the study.

Item	Content
In scope	Defects4J [25] (Java), 100 single-file bugs; two model tiers, by default two models of one family through an API; test-driven repair, where the failing test is known; the faulty methods are given to the model; success means a plausible patch
Measured	time, CPU time, tokens and money
Out of scope	other languages and benchmarks; finding the faulty code; judging whether a plausible patch is correct; issue resolution without a failing test (SWE-bench); training a router; more than two tiers; tests run in parallel on many machines

Two scope decisions deserve a word. Learned routers are out of scope as a technique but not as a comparison: the hindsight router of RQ3 is an upper bound on what any of them could save. And the bot assumes that a failing test exists, as it does in CI; without one a stall cannot be verified, and the rule does not apply.

1.4 Structure

The rest of the proposal is organised as follows.

- Chapter 2 gives the background: the cost of a tiered session and when escalation pays.
- Chapter 3 reviews related work and the open problems.
- Chapter 4 presents the tiered repair bot and its four modules.
- Chapter 5 plans the evaluation.
- Chapter 6 states how each possible result will be read.
- Chapter 7 gives the work plan, risks, threats to validity and future work.

Chapter 2

Background

2.1 APR Pipeline

Fault localisation finds the code most likely to hold the bug. The generator proposes a candidate patch there, and the validator builds it and runs the test suite. A plausible candidate goes on to a human or automatic correctness check. A failing one, in an LLM-based loop, returns its failure to the loop, which asks for the next candidate. A tiered loop makes one more decision in each round: which model proposes next.

Part of each validation is overhead: recompiling and starting the runtime. On the JVM it can be removed by validating in one long-lived JVM [8, 9]. The rest is running tests.

2.2 Cost Model of a Tiered Session

A session runs in rounds. In each round one model proposes candidates and the validator judges them. The session ends at the first plausible candidate or when its budget is spent. With two tiers its cost is

$$E \;=\; G_1 + G_2 + V, \tag{2.1}$$

where G_1 and G_2 are the cost of the model calls of the cheap and of the strong tier, and V is everything it costs to validate the candidates: applying the patch, building it, starting the runtime and running the tests. Cost is measured in three units, each reported on both sides: wall-clock time, which is what a user waits for; compute (CPU seconds for validation, tokens for generation); and money, from a fixed price list.

What the session spent. Splitting generation by tier and validation by step gives five components, each read directly from the session:

$$E \;=\; \underbrace{G_1 + G_2}_{\text{generation}} \;+\; \underbrace{V_{\text{build}} + V_{\text{start}} + V_{\text{test}}}_{\text{validation}}. \tag{2.2}$$

Each component is also split by how the session ended: repaired at the cheap tier, repaired at the strong tier, or not repaired.

What each family could save. Each family of techniques (Section 3.4) acts on part of this sum, and that part bounds what it can save (Table 2.1). The bounds are counterfactual. All of them are computed after the session from its record, without

running any technique.

Table 2.1: What each family of techniques can save at most.

Family	Upper bound on its saving
Execution acceleration	the build and start-up parts of validation, $V_{\text{build}} + V_{\text{start}}$
Test selection and prioritisation	the tests run before the first failing one, and the tests that do not reach the change
Verdict cache	the validation of repeated candidates
Model escalation	what the strong tier spends on bugs the cheap tier repairs, minus what the cheap tier spends
Early stopping	what is spent after the last round that made progress, in sessions that are never repaired

The share of each component is a property of a setting, not of a technique. Generation scales with the price and latency of the model, the build and start-up share with the runtime, and the rest with the tests. The same technique can therefore be worth deploying in one setting and not in another.

2.3 Escalation and Its Break-Even

This section states the problem that escalation solves. Fix a bug, and let the cheap tier propose up to r candidates. Let X be the position of its first plausible candidate, with $X = \infty$ if none of the r is plausible. Let c_1 be the cost of one cheap candidate, generation and validation together, and let S be the cost of a session of the strong model alone on the same bug.

Definition 2.1 (Cold escalation)**.** The session draws cheap candidates until one is plausible or r have failed. In the second case it starts the strong model as if nothing had happened: the strong model receives the same first prompt, the same feedback and the same budget as when it runs alone.

Under cold escalation, the strong part of the session is distributed exactly as a session of the strong model alone. The escalating session costs $c_1 \cdot \min(X, r)$, plus S when $X = \infty$. Two properties follow.

Proposition 2.2 (No lost repairs)**.** Under cold escalation, the session ends with a plausible patch whenever the strong model alone would have, on the same draws.

Proof. If a cheap candidate is plausible, the session has succeeded. Otherwise the session runs the strong model's session unchanged, and succeeds exactly when that session does. □

Proposition 2.3 (Break-even)**.** Cold escalation costs less in expectation than the strong model alone exactly when

$$\mathbb{E}\Big[S \cdot \mathbf{1}\{X \le r\}\Big] \;>\; c_1 \cdot \mathbb{E}\Big[\min(X, r)\Big].$$

Proof. The strong model alone costs S. Escalation costs $c_1 \cdot \min(X, r)$, plus S only when $X = \infty$. The difference is $S \cdot \mathbf{1}\{X \leq r\} - c_1 \cdot \min(X, r)$; take expectations. □

The left side is what the strong model would have spent on the bugs the cheap tier repairs. The right side is what the cheap tier spends on all bugs. A sufficient condition is simple to check. A strong session costs at least one strong call, c_2, so escalation pays whenever

$$\Pr[X \leq r] \; > \; r \cdot \frac{c_1}{c_2}. \tag{2.3}$$

In words: the cheap tier's success rate within its budget must exceed its budget times the price ratio. A condition of the same form is derived for routing by code health [26].

Example. Suppose a cheap candidate costs one thirtieth of a strong call, and the cheap tier may propose five. Escalation is then sure to pay if the cheap tier repairs more than 5/30, about 17 %, of the bugs. For comparison, low-cost models are reported to repair over half of moderately complex bugs [19]. If the strong model usually needs several calls, the threshold is lower still.

What the propositions do not cover. Four things, all of which are measured (Chapter 5).

- **Time.** On a bug the cheap tier cannot repair, escalation adds the time of r cheap candidates before the strong model starts.
- **A shared budget.** If the cheap tier's spending is taken out of one total budget, the strong model has less left, and Proposition 2.2 no longer applies.
- **An early stop.** A rule that ends the strong part before its budget is spent can lose a repair in the same way.
- **A warm handover.** If the strong model is given the cheap tier's evidence, its session is no longer distributed as when it runs alone. It may do better or worse.

2.4 Verified Progress and Stall

A rejected candidate leaves a *failure signature*: the test that rejected it, the exception type and its message, without line numbers. A candidate that does not compile leaves the first compiler error as its signature. A candidate makes *progress* when it is new and fails in a new way. A repeat, or new code that fails exactly as an earlier candidate did, makes none. A session *stalls* when its latest candidates make no progress; Sections 4.5 and 4.6 fix the exact rule for each tier.

Progress and stall are *verified*: they are read from executed tests, not predicted from the prompt or from the model's own account. This matters for two reasons. A prediction can be wrong in either direction: it can keep a session going that the tests have already refuted, or reject a patch that the project's tests accept, as when a model judges its patch by tests it wrote itself. And a verified signal needs no training data, so it is available on the first bug of a new project.

2.5 What Can Be Handed Over

When a session moves to another model, three kinds of history can move with it:

- **Reasoning**: the first model's explanations of the bug.
- **Drafts**: its candidate patches.
- **Verified evidence**: which tests rejected which behaviour, and how.

The first two are the first model's opinions. The third is a fact about the program. A handover that passes only verified evidence can be shorter than a transcript, and it does not ask the next model to trust the first. Whether it helps is an empirical question (RQ4).

2.6 Benchmark

Repair tools are evaluated on benchmarks of real bugs, each shipped with tests that expose the bug. This thesis uses **Defects4J** [25], the common benchmark of Java repair: real bugs from open-source projects, each with the developer's fix and at least one test that fails before the fix and passes after it. The on-the-fly validators and the study of test selection in repair were evaluated on it [8–10], and so was conversational repair [3].

Three properties of a benchmark decide how much escalation can help: the share of bugs the cheap model repairs, the price ratio between the tiers, and what one validation costs. The first is likely inflated on Defects4J, since current models may have memorised it [27]. Section 5.1 handles this by choosing the model pair on a development set with a rule fixed in advance.

Chapter 3

Related Works

Related work falls into three parts: the LLM-based repair systems that generate and validate candidate patches; the studies that measure what such systems cost; and the techniques that make a repair loop cheaper. The techniques are grouped into seven classes by the part of the cost they reduce. Model selection, the group this thesis works on, is reviewed in depth; the others briefly.

The review is a structured literature review, not a systematic one: a keyword search of the major digital libraries, software-engineering venues and arXiv for LLM-based repair, repair cost and model routing, followed by backward and forward snowballing from four seed papers [3, 16, 28, 29]. The search was last run in November 2026, and several of the closest works are preprints of that year.

3.1 LLM-Based Repair Systems

A recent survey classifies LLM-based repair systems by how much control the model has over the repair loop [5]. Table 3.1 groups representative systems into three families.

Table 3.1: Representative LLM-based repair systems, what they optimise, and how they use validation.

Family	Systems	Optimises	Use of validation
Conversational	ChatRepair [3]	patch quality from failure feedback, at low cost per bug	every candidate is tested; failures return to the conversation
Autonomous agents	RepairAgent [4], SWE-agent [30], AutoCodeRover [31]	localisation and context gathering through tools	the agent decides when to run tests
Agentless pipelines	Agentless [32]	a fixed localise–repair–validate pipeline with low, predictable cost	selected regression tests and generated reproduction tests filter and rank candidates

Conversational and pipeline repair. ChatRepair [3] alternates generation and validation in one conversation: earlier incorrect patches and their test failures stay in the prompt, so that the model can avoid repeating them. It reported fixing 162 of 337 bugs at about \$0.42 per bug, yet 65 % of the patches it generated were duplicates of earlier ones [13]. Agentless [32] replaces free choice of actions with fixed phases, which gives predictable cost and far fewer tokens than tool-driven agents [5].

Agent-based repair. Autonomous agents such as RepairAgent [4], SWE-agent [30] and AutoCodeRover [31] let the model choose tools to read, edit and test code. They improve localisation and context at the price of long sessions; RepairAgent spends 99 % of its repair time executing tools [4].

One model per session. As far as their papers report, each of these systems runs one model for the whole session, chosen before the bug is seen. Which bugs needed that model is not reported.

3.2 Validation, Repeats and Stopping in Current Tools

The papers of these systems describe how a candidate is validated and when a session stops. They say much less about a candidate that was proposed before, or about a session that goes nowhere.

Validation. ChatRepair and RepairAgent run the project's tests on each candidate and return the failure to the model [3, 4]. Agentless validates after generation: it filters its samples with regression and reproduction tests and then votes among them [32]. In SWE-agent and AutoCodeRover the model decides when a test is run [30, 31]. Deployed systems put further checks between the tests and the developer who reviews the fix [33–35].

Stopping. Each of these systems is described with a limit that is set in advance: a number of attempts, cycles or steps, or a cost per bug. The session ends when the limit is reached or a patch passes.

What the descriptions leave out. As far as these papers report, none describes reusing the verdict of a candidate that was already tested, stopping because the tests show no progress, or reporting why a session gave up. Repeats are common where they have been counted: 65 % of the patches of ChatRepair duplicate earlier ones [13]. A repeat, a stall and a give-up are the events that the Evidence Memory, the Escalation Controller and the Budget Guard act on.

3.3 Cost Measurement in LLM-Based Repair

Studies that report what LLM-based repair costs fall into four kinds.

Cost per bug, as a by-product. Repair systems report a cost next to their success rate, each in its own unit: ChatRepair in API dollars per bug [3], RepairAgent in tokens and in repair time per bug [4], and CigaR, which makes token cost its objective, in tokens per bug: 127k against 467k for its baseline, a reduction of 73 % [36]. These figures compare systems on one side of the bill and are not normalised across studies [5].

Cost-aware comparisons. A second kind holds the budget fixed and asks which strategy spends it better. Independent repeated prompting repairs more bugs than a conversational repairer at the same number of attempts [13]. For software agents,

SWE-Effi re-scores SWE-bench systems by their resolve rate per token and per unit of time, and finds that unresolved issues consume far more resources than resolved ones [21].

The cost of execution. Lin et al. [28] isolate execution in agents. Every agent they examine relies on code execution, but applies it indiscriminately: for commercial agents on SWE-bench, prohibiting execution changed the resolve rate by only 1.25 percentage points, while saving 56 % to 62 % of tokens and 48 % to 54 % of wall-clock time for one of them.

Cheap against expensive models. Recent studies compare models of different price on the same bugs. Liu et al. [19] run two repair techniques with three model families: low-cost models repair over half of the moderately complex bugs, and higher-cost models or stronger reasoning settings do not consistently improve cost-efficiency. Sun et al. [20] find that smaller models produce correct patches that no other model finds, and that most correct patches appear early among the samples. Vallecillos-Ruiz et al. [37] find that quantised and full models repair different bugs with little overlap, and that the quantised models take more inference time and energy.

What is not measured. These studies compare whole models or whole sessions. None splits the cost of one test-driven loop by model tier and by validation step, and none reports how much of the strong model's spend falls on bugs that a cheaper model repairs. Without that split, the most that escalation could save is unknown.

3.4 Classification of Cost-Reduction Techniques

Table 3.2 sorts the techniques into seven groups by the part of the cost they cut. Because the groups cut different parts, they can be combined. Groups A to D act on validation; the system of Chapter 4 provides them with existing techniques. Groups E to G act on generation; this thesis works on F and uses a simple rule for G.

3.5 Model Selection: Routing and Cascades

Principle. A cheap model answers many inputs as well as an expensive one. *Routing* sends each input to one model, chosen before any model runs. A *cascade* runs the cheap model first and calls the next model only when the cheap answer is judged not good enough. Both need a judge, and the judge is where the approaches differ.

Key techniques. Table 3.3 sorts them by what the judge reads.

FrugalGPT introduced the LLM cascade: a learned scorer judges each answer, and the cascade is reported to match the best single model at up to 98 % lower cost [29]. RouteLLM learns a router from preference data and reports savings of over two times in some cases [39]. For code, Chen et al. [22] let each model write tests for its own solution and step up when they fail, which lowers cost by 26 % on average.

In issue resolution, Gandhi et al. [23] compare strong-weak collaboration strategies

Table 3.2: The seven groups of cost reduction.

Group	Cuts	Signal	Needs training	Representative work
A test selection	tests that do not reach the change	coverage	no	RTS in APR [10], PIT [12]
B test prioritisation	tests run before the first failing one	failure history, test cost	no	FRTP [11]
C execution acceleration	build and start-up	none	no	UniAPR [8], ExpressAPR [9]
D patch reduction	validation of repeated or equivalent candidates	patch text, test behaviour	no	verdict cache, test-equivalence [38]
E generation-side reduction	tokens per call, and rounds	prompt design	no	ChatRepair [3], CigaR [36]
F model selection	strong-model calls on bugs a cheaper model repairs	prompt, trajectory, self-written tests	usually	FrugalGPT [29], SWE-Router [16]
G early stopping	rounds after the session is lost	trajectory, hidden states, experience	yes	EET [24], FailFast [18]

Table 3.3: Model selection, by the signal the judge reads.

Signal	Rule	Representative work
A learned score of the answer	cascade: stop at the first answer scored reliable	FrugalGPT [29]
Human preference data	route by a learned win probability	RouteLLM [39]
Tests the model writes for itself	cascade: step up when the cheap model's code fails its own tests	Model Cascading for Code [22]
Patch format	the weak model tries n times; the strong one is called if no patch is well-formed	Weak LM First [23]
Code health metrics	route up front to the cheapest tier predicted to pass	Triage [26]
The first turns of the cheap agent	a fine-tuned classifier decides at a checkpoint whether the strong model takes over	SWE-Router [16]

on SWE-bench Lite. Their *Weak LM First* is the closest to this proposal: the weak model makes n attempts, and the strong model is called when none yields a valid patch. Its trigger is the format of the patch, not a test. Their most effective strategy, in which the strong model drafts and the weak one refines, matches the strong model at 40 % lower cost. SWE-Router [16] lets a cheap agent run a few turns; a fine-tuned classifier then reads that partial trajectory and decides whether the strong model takes over. On escalation the strong model starts fresh, on the stated ground that the weak model's reasoning biases it. Triage [26] routes up front on code-health metrics, and derives a break-even condition of the same form as Proposition 2.3.

Limitations. Four limits matter here.

- **The judges predict.** Scorers, classifiers and self-written tests can be wrong, and

a wrong self-written test can reject a patch that the project's own tests accept.

- **Most judges are learned.** They need trajectories from the same kind of task before the first decision.
- **Only tokens are counted.** None of these studies prices the validation of the extra cheap candidates, which on a cold path costs a build and a start-up each (Section 4.3).
- **The handover is assumed, not measured.** SWE-Router discards the cheap model's work because it may bias the strong model [16]. What a verified record of failures is worth to the next model has not been measured.

3.6 Other Cost-Reduction Techniques

A: Test selection. A test that never executes the code a patch modifies gives the same result on the patched program as on the original, so it can be skipped. Lou et al. [10] measured regression test selection inside twelve repair systems on more than two million patches: every granularity substantially improved efficiency, finer granularity was better, and combining selection with prioritisation was best.

B: Test prioritisation. Validation can stop at the first failing test, so the order of the tests decides how much is spent before it. FRTP runs first the tests that rejected earlier candidates of the same session [11]. The mutation tool PIT runs only the tests that cover the mutated line, fastest first [12]. This proposal adopts both rules as they are, and measures what they leave to any other order (Section 5.4).

C: Execution acceleration. Part of a test run is overhead: recompiling the patched program and starting the runtime. UniAPR keeps one JVM alive and reloads only the patched classes, which speeds repair up by over an order of magnitude [8]. ExpressAPR borrows five accelerations from mutation testing and validates 137 times faster than plain validation [9]. Both are built for the JVM and were measured on patches produced by search, before LLM-based repair.

D: Patch reduction. Some candidates need not be run at all. A verdict cache gives a repeated candidate its stored verdict, which matters for LLMs, since 65 % of the patches of ChatRepair were duplicates [13]. Test-equivalence analysis runs one candidate per group of candidates that behave alike on the tests [38].

E: Generation-side reduction. A loop that needs fewer rounds, or fewer tokens per round, is cheaper on both sides. The levers are what the model is shown after a failure [3], many candidates per call to amortise the prompt [36], and independent re-sampling instead of conversational refinement [13]. Small models in particular are hurt by their own history. Shown its failed attempt, a model of up to 7B parameters reproduces a near-identical program in 33 % to 68 % of retries, against 2 % to 14 % under blind re-sampling, and blind re-sampling is the strongest retry condition below 7B [14]. Olausson et al. [15] find that the gains of self-repair are often modest once the cost of the extra calls is counted. REx chooses which earlier candidate to refine with

Thompson sampling, and solves more problems with fewer model calls [40].

G: Early stopping. Failed sessions are the expensive ones [21], and three recent methods stop them early. EET extracts experience from earlier runs and lowers cost by 32 % on average, for a loss in resolve rate of at most 0.2 % [24]. FailFast trains a small monitor on agent trajectories and saves 14.6 % to 20.4 % of execution tokens. Its companion, RestartSmart, restarts the same agent with the interrupted diff as an optional overlay, and lifts the resolve rate from 66.6 % to 71.8 % where a cold restart reaches 66.8 % [18]. Ruan et al. [41] stop episodes from linear probes on the model's hidden states. All three learn their signal, from earlier trajectories or from the model's internals.

3.7 Research Gap

Three observations close the review. First, the validation-side groups have techniques with measured savings [8–10]; a repair bot can adopt them, and the model call is what they leave. Second, model selection and early stopping are being studied intensively, but for issue-resolution agents, with learned judges, and in tokens. Third, test-driven repair differs from that setting in the one respect that matters to a judge: the tests are there. The cheap attempt is itself a measurement of difficulty, and its outcome is verified.

Open problems. Together, the observations give five open problems, which the proposed system addresses (Chapter 4).

O1 The cost of a test-driven repair loop is not split by tier. Without the split, the most that escalation can save is unknown.

O2 Escalation is triggered by predictions. No study uses verified test outcomes as the trigger, or compares a trigger with a hindsight bound in this setting.

O3 What to hand over is unresolved. Reasoning and drafts are reported to bias the next model [16, 17]. An optional diff helped a restarted agent [18]. Verified evidence has not been separated from the rest.

O4 The validation of cheap candidates is not priced. Cheap-first multiplies candidates. On a cold path each costs a build and a start-up, which can cancel the saving in time.

O5 Repeats and stuck sessions are not managed. Small models anchor on their own attempts [14], a conversational loop repeats most of its patches [13], and the systems of Section 3.2 are described as stopping at a limit fixed in advance.

Chapter 4

Proposed System

This chapter presents the proposed system, a *tiered repair bot*. Section 4.1 states what a practical repair bot needs. Section 4.2 shows the bot. The next four sections describe its modules, and Section 4.7 states when it helps.

4.1 Usage Scenario and Requirements

Scenario. A test fails in continuous integration. A repair bot is given the faulty code and tries to fix it within a budget set by the team. It returns either a plausible patch for a developer to review, or a report of why it gave up. Developers name CI as the place they most want repair to run [6], and deployed systems work this way: they act on failures found in CI and send every fix to a reviewer [33, 34, 42]. Because it runs on every failing build, the bot meets easy and hard bugs alike, and it cannot know in advance which is which. Table 4.1 lists what such a bot needs.

Table 4.1: What a repair bot in CI needs, and the evidence.

Requirement	Evidence	Served by
P1 bounded time	63 % of 103 practitioners would not wait over an hour [6], though correctness matters more than speed [43]	M0 Warm Substrate, M1 Evidence Memory
P2 predictable, low cost	budgets decided the adoption of repair at Bloomberg [42]	M2 Escalation Controller, M3 Budget Guard
P3 drop-in	developers want repair inside the CI pipeline [6], fitting their workflow [44]	no training data; one coverage run of the faulty program
P4 trustworthy	a fix must pass all CI checks and be reviewed [42]	M0 and M1: every verdict comes from the tests
P5 explain a failure	unexplained fixes are ignored [44]; developers value the fault location and tests [6]	M3 Budget Guard

The evidence is strong for P3 and P4 and mixed for P1, so the bot aims at a bounded time, not the shortest. P2 and P5 are supported only indirectly; they are design choices the evidence favours.

4.2 System Overview

Figure 4.1 shows one session. Before the first round the **Warm Substrate** (M0) starts the test runtime on the project as the failing build left it; the runtime then stays alive for the whole session. The cheap model proposes candidates. The **Evidence Memory** (M1) checks each one: a candidate seen before gets its stored verdict, and a new one is compiled by the substrate, loaded into the live runtime and run against the scheduled tests until the first failure. After a rejection, the **Budget Guard** (M3) checks that budget is left, and the **Escalation Controller** (M2) checks whether the candidate made progress. While the cheap tier makes progress, it proposes again. When it stalls, the controller writes a handover note and the strong model takes over. If the strong tier stalls too, or the budget is spent, the session ends with a give-up report.

The repair loop interacts with the bot through a single call. It takes a candidate and returns the verdict, the evidence, and what the session should do next:

```
result = bot.validate(candidate)
result.verdict    # plausible, rejected, does not compile, or repeat
result.evidence   # the failing test and its failure signature
result.next       # same tier, step up, or stop
result.note       # the handover note, when next is step up
result.report     # the give-up report, when next is stop
```

The bot is configured with a list of tiers and a budget per bug. It needs no training data.

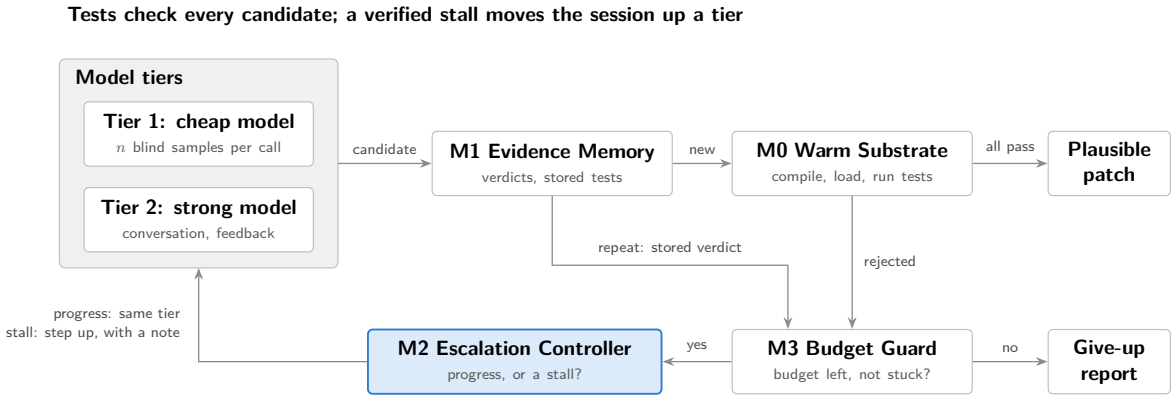


Figure 4.1: One repair session with the tiered bot. Every candidate of either tier passes through the Evidence Memory and the Warm Substrate. The Escalation Controller (highlighted) is the new module: while a tier makes progress it proposes again, and a verified stall moves the session up a tier with a handover note.

A session, step by step. Table 4.3 follows one bug through the bot.

The strong model was called twice. The ten cheap candidates cost two calls of the cheap model and eight validations on the warm substrate; two of them were answered from the memory.

Table 4.2: The four modules: what each improves, the cost it targets, and how.

Module	Improves	Targets	How
M0 Warm Substrate	the build and the start-up are paid once per session, not once per candidate	the build and start-up parts of validation, and the tests that do not reach the change	in-process compilation of the patched file; a fresh class loader in a live runtime; a fixed test schedule
M1 Evidence Memory	nothing is validated twice, and evidence survives a model switch	the validation of repeats and of candidates a stored test refutes	normalised verdict cache; stored failing tests and failure signatures
M2 Escalation Controller	the strong model is called only when it is needed	strong-tier calls on bugs the cheap tier repairs	cheap tier first; Step moves up on a verified stall; handover note
M3 Budget Guard	predictable cost, and a reason when it fails	sessions that are never repaired	a budget per tier and per session; a stop when the top tier stalls; give-up report

Table 4.3: One session of a bug in `DateUtils` (illustrative).

Step	Tier	Candidates	Evidence Memory	Controller
1	cheap	five from one call	two are the same patch after normalising; of the four distinct ones, one does not compile, two fail `DateUtilsTest` with error A and one fails `CalendarTest` with error B	progress: three new signatures
2	cheap	five more	one repeats a patch of step 1 and is answered from the memory; four are new, and all fail with error A or B	no new signature: stall, step up
3	strong	one, given the handover note	fails `DateUtilsTest` with a new error C	progress
4	strong	one, given the failure	passes every test	plausible: stop

4.3 Module M0: Warm Validation Substrate

Principle. During a repair session the project, its dependencies and the test runtime never change. One file changes, in the bodies of a few methods. A validator that rebuilds the project and starts a fresh runtime for every candidate pays, once per candidate, for work whose result is the same every time. A cheap-first bot multiplies candidates, so it pays this more often than any other design.

Mechanism. M0 does this work once per session and keeps it.

1. **Build once.** The faulty program is compiled once, with the project's own build. In CI the failing build has already done this.
2. **Compile one file, in process.** For each candidate only the patched source file is compiled, by the compiler shipped with the runtime (`javax.tools`), inside a process that stays alive, against the classes of the first build.
3. **Load, do not restart.** The new bytecode is loaded into the live test runtime

through a fresh class loader, and the tests are run there. The Java runtime and the test framework stay loaded and warm; the classes of the project, its tests and its libraries are loaded again for each candidate.

Why a fresh class loader. On-the-fly validators for classical repair redefine the patched class in place [8]. Redefinition has two limits in this setting. It cannot change the shape of a class, and a candidate that only rewrites a method body changes it as soon as the new body holds a lambda or an anonymous class. And it keeps the static state of every loaded class, so what one candidate leaves behind has to be reset by other means: UniAPR re-runs class initialisers between patches, and before it did so it disagreed with ordinary validation on 4 of 74 bugs [8]. A class loader per candidate gives both properties by construction. Any candidate that compiles can be loaded, and no static state of the project, its tests or its libraries survives from one candidate to the next. The price is that these classes are loaded again each time; the build and the start-up are still paid once.

Calibration. At the start of a session the substrate compiles the unchanged file in the live runtime and runs the scheduled tests once. Every test must end as it did in the failing CI run: the tests that failed there fail, and the others pass. If one differs, the live runtime is not trusted for this bug, for instance because a test depends on how its classes are loaded, and the session falls back to the cold path. The run also warms the runtime up before the first candidate arrives.

Test schedule. The substrate runs the tests in a fixed order that needs no learning: first the tests that failed on the faulty program, then the tests that rejected earlier candidates of the session, then the remaining tests that cover the faulty methods, fastest first. Tests that do not cover the faulty methods are skipped, which is safe when a patch only replaces the bodies of those methods. The first step is common practice, the second is FRTP [11], and the third is the rule of PIT [12]. What this schedule leaves to any other order is measured (Section 5.4).

Reported failure. A candidate can fail several tests, and the order decides which failure is met first. What is reported about a rejected candidate, to the model and in its failure signature, is therefore fixed by the static part of the order: its first failing test among those that failed on the faulty program, and otherwise among the covering tests, fastest first. The tests that rejected earlier candidates change what a rejection costs, never what is reported. A candidate is thus told the same thing whatever was tried before it, and this is what allows a session to be replayed (Section 5.5). A deployed bot, which stops at the first failure it meets, can differ from this rule only on a candidate that passes every originally failing test and then fails several others; how often that happens is measured.

Safety. M0 must not change any verdict. Three things could.

- **State that outlives a class loader**: system properties, the default locale and time zone, files, threads. The substrate restores the first three after each candidate and interrupts the threads a candidate leaves behind.

- **A candidate that hangs or kills the runtime.** A test that passes its time limit cannot be stopped inside a JVM, and a candidate may exit the process. The substrate counts that test as failed, discards the runtime and starts a new one, and the new start is charged to the candidate.
- **Tests that depend on how classes are loaded.** The calibration run catches them before the first candidate.

The question is treated as empirical. Every candidate of the study is validated on the cold path as well, the verdict of record is the cold one, and every disagreement is reported (Section 5.3).

Expected size. Table 4.4 sets the two paths side by side. UniAPR and Express-APR report speed-ups of one to two orders of magnitude on classical repair tools [8, 9], where most of what they remove is this same build and start-up. How much of that carries over to an LLM loop, which validates tens of candidates per bug rather than thousands, is what RQ2 measures. For escalation the consequence is direct: on the cold path every extra cheap candidate pays a build and a start-up of its own.

Table 4.4: What one candidate pays with and without M0.

Step	Without M0 (cold path)	With M0 (warm path)
Build the candidate	the project's own build	one file, compiled in process
Start the test runtime	once per candidate	once per session, with the calibration run
Run the scheduled tests	to the first failure	the same tests, in a runtime that is already warm
Candidate that hangs or kills the runtime	the runtime is discarded anyway	a new runtime is started and charged to the candidate

Relation to existing work. On-the-fly patch validation is not new [8, 9]. M0 claims no new mechanism. Its role here is to make the cheap tier's candidates cheap to check, and its contribution is the measurement of what that is worth in an LLM loop (RQ2).

4.4 Module M1: Evidence Memory

Principle. What the tests have shown about a candidate is true whichever model proposed it. The memory keeps it for the whole session and across the switch between models.

What it keeps.

- **Verdicts.** Each candidate is first *normalised*: it is parsed into a syntax tree, comments are removed, local variables are renamed in order of appearance, and the tree is printed back. A candidate whose normalised form is already in the memory is a *repeat*: it gets the stored verdict without running any test.

- **Failing tests.** Every test that has rejected a candidate is kept and scheduled first for later candidates.
- **Failure signatures.** Each distinct signature (Section 2.4) is kept with a count and one example patch. The controller reads progress from them, and the handover note and the give-up report are written from them.

Across tiers. The memory outlives the switch. A strong-tier candidate that equals a cheap-tier one is answered from the memory, and one that fails a stored test is rejected by that test first. How often the strong model repeats what the cheap one already showed to be wrong is itself a measurement (RQ4). One rule keeps the tiers separable: a model is told when it repeats *itself*, but a repeat of the other tier's candidate is reported to it as an ordinary failure. Under a cold handover the strong model is therefore shown exactly what it would be shown if it ran alone.

Safety. Normalisation removes only differences that cannot affect behaviour: layout, comments and the names of local variables. Two patches therefore share a verdict only when they behave identically, so reusing it is safe for deterministic tests. Running stored failing tests first changes the order of the tests, not the set, so it never changes a verdict.

4.5 Module M2: Escalation Controller

Principle. A bug should be repaired by the cheapest model that can repair it. Which model that is cannot be known in advance, but it can be found out cheaply: the cheap model tries first, and the tests say whether it is getting anywhere.

Tier 1: blind re-sampling. The cheap tier does not hold a conversation. Each call sends the same first prompt and asks for n candidates; nothing about earlier failures is added. Two results support this choice. Independent re-prompting repairs more bugs than a conversation at the same number of attempts [13]. And small models anchor on their own failed attempt [14]. Blind re-sampling also keeps the prompt constant, so one call returns several candidates for the price of one prompt [36].

Tier 2: conversation with feedback. The strong tier runs a conversational loop in the style of ChatRepair [3]: after each rejected candidate, the failing test and its error are fed back. What such feedback gains depends on the model that reads it [15], which is why only the strong tier receives it.

The rule. After each cheap call, the controller counts the candidates that made progress (Section 2.4). A call has *stalled* when at most one of its candidates made progress. The session steps up after a stalled call, or when the cheap tier's budget of candidates is spent. On a step-up the controller writes the handover note and starts the strong tier. Algorithm 1 gives the rule, called Step. The candidates of a call are validated in the order they were returned, and the session ends at the first plausible one.

The handover note. The note is written by the bot from the Evidence Memory,

Algorithm 1 Step for one session

Require: cheap model m_1, strong model m_2; n candidates per cheap call; cheap budget B_1 candidates; strong budget R_2 rounds

$seen \leftarrow \emptyset$ ▷ failure signatures, kept by the Evidence Memory
while fewer than B_1 cheap candidates were drawn **do**
  draw n candidates from m_1 with the first prompt
  $new \leftarrow 0$
  for each candidate p **do**
    $v \leftarrow$ VALIDATE(p) ▷ the memory, then the schedule
    if v is plausible **then return** p
    if signature(v) $\notin$ *seen* **then** add it to *seen*; $new \leftarrow new + 1$
  end for
  if $new \leq 1$ **then break** ▷ the call stalled
end while
$note \leftarrow$ HANDOVER(*seen*) ▷ empty under a cold handover
for $round = 1$ **to** R_2 **do**
  $p \leftarrow$ next candidate from m_2, given *note* and the feedback so far
  $v \leftarrow$ VALIDATE(p)
  if v is plausible **then return** p
  if the Budget Guard stops the session **then break** ▷ Section 4.6
end for
return the give-up report

not by the cheap model. It holds the number of candidates tried; each distinct failure signature, most frequent first, with its test, its error and the number of candidates that ended there; and the compiler errors seen. It holds none of the cheap model's explanations. Figure 4.2 shows an example. Four handovers are compared (RQ4):

- **Cold**: no note. The strong tier starts from the first prompt alone.
- **Evidence**: the note described above.
- **Evidence and patches**: the note, with the shortest candidate that produced each signature.
- **Transcript**: every cheap candidate and its failure, in the order they occurred.

```
Handover note (10 candidates from the first model, none plausible)
Verified failures, most frequent first:
  6x DateUtilsTest.testTruncateLang59
      AssertionFailedError: expected <...MDT 2004> but was <...MST 2004>
  2x DateUtilsTest.testRound
      ArithmeticException: Calendar value too large
  1x compile error: cannot find symbol
1 candidate repeated an earlier one. No candidate passed DateUtilsTest.
```

Figure 4.2: An example handover note (illustrative values).

Guarantee. Under a cold handover, with R_2 equal to the budget the strong model has when it runs alone, and with the Budget Guard off, Proposition 2.2 applies: the

session repairs every bug that the strong model alone would repair on the same draws. The price is the cheap tier's cost and delay on the bugs it fails, which Proposition 2.3 weighs against the saving. A note gives up this guarantee for a possible gain, and the guard gives it up for a saving. That is why both effects are measured rather than assumed, and why a cold handover without a stop is the default until they are.

Expected size. Three figures bound the expectation. For a prompt of 1,500 tokens and candidates of 400 tokens, a cheap call that returns five candidates costs about one sixth of one strong call at list prices [7], so one cheap candidate costs about one thirtieth. This counts no reasoning tokens: the cheap tier runs without reasoning, and the reasoning of the strong tier only lowers the ratio. By the sufficient condition of Section 2.3, ten cheap candidates then pay for themselves if the cheap tier repairs more than a third of the bugs, and five if it repairs more than a sixth. And in neighbouring settings escalation saved between 26 % and 40 % of cost [22, 23]. The expectation is therefore a saving of tens of percent in money, not a factor. The experiments report it together with the break-even it implies.

Relation to existing work. Cascades and routers are not new [16, 22, 23, 26, 29, 39]. Three things differ here. The trigger is a verified stall rather than a prediction, so it needs no training and cannot be wrong about a verdict. The cheap tier's candidates are validated on a warm substrate, and that validation is priced. And the handover is varied and measured instead of fixed.

4.6 Module M3: Budget Guard

Principle. The Budget Guard keeps the cost per bug within a budget and reports why a session stopped. Failed sessions account for most of the cost [21], and the systems of Section 3.2 are described as stopping at a limit fixed in advance.

When to stop. Each tier has its own share of the session: B_1 candidates for the cheap tier and R_2 rounds for the strong tier. On top of these the guard watches two things. It stops the session when the top tier stalls, that is, after r consecutive rounds of the strong tier without progress, where progress is judged against everything the session has seen. And it stops before a model call that the remaining budget B, in time, tokens or money, cannot pay for, judged by the average cost of the tier's calls so far. The values of r and B are chosen on the development set (Section 5.10).

Give-up report. A session that ends without a plausible patch returns: why it stopped; what each tier tried and what it spent; and the failure signatures, most frequent first, with the test, the error and an example patch. The signatures show where the models repeatedly failed, which is the kind of information (fault location and tests) that developers find most useful when they fix a bug themselves [6]. Figure 4.3 shows an example.

Measured apart. Stopping is the one decision of the bot that can lose a repair the strong model would still have made. In the experiments the guard therefore only records when it would stop. The stop is applied afterwards by replay, which gives its

```
Give-up report (bug Lang-65, session 2)
Stopped: no progress for 3 rounds at the strong tier
Tiers:   cheap 10 candidates (9 distinct, 1 repeat); strong 6 rounds
Spent:   68 s (models 61 s, validation 7 s), 33k tokens, $0.07
Failures, most frequent first:
  9x DateUtilsTest.testTruncateLang59
      AssertionFailedError: expected <...MDT 2004> but was <...MST 2004>
  5x DateUtilsTest.testRound
      ArithmeticException: Calendar value too large
  1x compile error: cannot find symbol
Example patch for the most frequent failure (strong tier, round 4):
    private static void modify(Calendar val, int field, boolean round) {
        ...
    }
```

Figure 4.3: An example give-up report (illustrative values).

saving and its loss for every value of r and B from the same sessions.

4.7 When the System Helps

- **Escalation** saves most when the price gap between the tiers is wide and the cheap tier repairs a fair share of the bugs. If it repairs almost none, the bot should start at the strong tier. If it repairs almost all, the strong tier is rarely called, and the saving approaches the whole gap.
- **The Warm Substrate** saves in proportion to the share of validation spent building and starting. It is also what keeps cheap-first from costing time.
- **The Evidence Memory** saves in proportion to how often the models repeat themselves and each other.
- **The handover note** can help only on bugs that reach the strong tier and are not repaired in its first round.
- **The Budget Guard** saves most when sessions go on long after the models are stuck.
- **A slow strong model** is the one cost that no module removes. Escalation only avoids calling it.

The modules therefore help in different regimes, and the regime decides their order of importance rather than the other way round. RQ1 says which regime the benchmark is in.

Chapter 5

Evaluation Plan

The evaluation has two levels (Table 5.1). Policies that do not change what a model is shown are compared by *replay*: sessions are recorded once, and every policy is charged from the same recorded candidates. Handovers that do change what the strong model is shown are run live.

Table 5.1: The five experiments.

Experiment	RQ	What is run
E0 the warm substrate	RQ2	every distinct candidate of the study validated twice, once on the cold path and once on M0: cost per step and any disagreement in verdict; no model call
E1 single-tier sessions, live	RQ1	each model alone on the 100 evaluation bugs, 3 sessions per bug: the cheap model to 20 candidates, the strong model to 10 rounds
E2 escalation policies, replayed	RQ3	six policies charged from the recorded sessions of E1; no model call
E3 handover, live	RQ4	on 50 bugs, the strong tier restarted with each of three notes in the sessions that step up
E4 whole system, live	RQ5	the full bot against the strong model alone on the other 50 bugs

5.1 Bugs and Setup

Bugs. Defects4J [25] version 3.0.1 has 854 real Java bugs from 17 projects. A bug is used if (i) it can be reproduced, that is, its failing tests fail before the fix and pass after it; (ii) the fix changes one source file, and in it only the bodies of existing methods; and (iii) its test suite has at least ten test classes and runs in under 5 minutes. From these, 120 bugs are drawn at random, spread evenly over the projects. Twenty form a *development set*, used to check the pipeline and fix every setting in advance. One hundred form the *evaluation set*, on which nothing is tuned; it is split at random into two halves of 50 for E3 and E4.

Faulty code given. Finding the faulty code is not studied. As is common in LLM-based repair [3], the model is given the methods that the developer's fix changes, and every candidate replaces these methods and nothing else. This favours repair success [45] but is the same for every configuration.

Model pair. Two tiers of one family through an API: GPT-6 Luna as the cheap tier and GPT-6 Sol as the strong tier, at the dated snapshots in use when the runs start.

Both models reason before they answer, by default at effort *medium*, and reasoning tokens are billed as output [46]. The strong tier keeps that default. The cheap tier runs with reasoning off: its part is to supply many cheap, independent candidates, and reasoning would raise the price of each one and slow the tier that every session waits for first. Whether this costs the cheap tier too many repairs is checked on the development set (Table 5.6). Defects4J is likely memorised by current models [27]. If the cheap tier repaired nearly every bug, or the strong tier repaired every remaining bug at once, there would be little to measure. The pair is therefore confirmed on the development set by a rule fixed in advance. The cheap tier must repair between 20 % and 80 % of the development bugs within its budget, and the strong tier must need more than one round on at least a third of the bugs the cheap tier fails. If the cheap tier repairs more than 80 %, or the strong tier fails the second condition, the pair moves down one step: a local 7B model (`qwen2.5-coder:7b`) becomes the cheap tier and Luna the strong one. If the cheap tier repairs less than 20 %, the pair is kept and the result is read as in Chapter 6. The thesis reports which pair was used and why.

Unit of testing. Tests are scheduled by test class. A candidate is plausible when it passes every scheduled class.

5.2 What Is Compared

Six policies are compared in E2 (Table 5.2). All of them use the same validation layer, so they differ only in which model is asked.

Table 5.2: The policies compared.

Policy	Rule	Role
Cheap-only	the cheap model, up to 20 candidates	reference
Strong-only	the strong model, up to 10 rounds, conversational	baseline
Fixed-5	step up after 5 cheap candidates	baseline
Fixed-10	step up after 10 cheap candidates	baseline
Step	Algorithm 1: step up after a stalled call, with at most 20 cheap candidates	proposed
Hindsight	for each session, the switch point that costs least, chosen with knowledge of the outcome	bound

Hindsight is not a policy that can be deployed. For each session it picks, among the switch points of 0, 5, 10, 15 and 20 cheap candidates, the one whose session costs least in money, and it prefers a switch point that ends with a plausible patch to one that does not. It therefore marks the most that any router could save by choosing when to switch, the learned routers of Section 3.5 included. The distance between Step and Hindsight is the room left for a better trigger.

Two readings of the budget. In the main reading, every escalating policy gives the strong tier its full 10 rounds, so Proposition 2.2 applies and no repair is lost. In a

second reading the total number of attempts is held at 10: five cheap candidates, then five strong rounds. There the repair rate can fall, and it is tested.

Handovers (RQ4). Cold, Evidence, Evidence and patches, and Transcript (Section 4.5), each with Step.

Variants of Step. A stricter stall, in which a call has stalled only when none of its candidates makes progress, is compared by replay. Three variants change what the cheap model is asked, so they are run live, on the development set only: other numbers of candidates per call (1, 3 and 10), a conversational cheap tier instead of blind re-sampling, and a cheap tier that reasons at effort *low*.

5.3 How Cost Is Measured

Audit. Every distinct candidate is compiled once and run on every scheduled test class, on the cold path and on the warm path, recording the result and time of each class. From this record the validation cost of any policy on that candidate is computed: compilation, start-up where it applies, and the classes run up to the first failure, in the order the session would have used. All policies are thus charged from the same measurement. The verdict of record is the one of the cold path.

How time is measured. On the cold path the classes of a candidate run one after another in a JVM started for that candidate, as build tools run a test suite; on the warm path they run in the live runtime. The runner records for each class its result, its wall-clock time and the CPU time of the JVM process while the class runs. A class that runs longer than 60 seconds counts as failed. The main unit for validation is CPU seconds, which depend less on the load of the machine than wall-clock time; wall-clock time is recorded as well. A session on the warm path is also charged, once, for starting the runtime and for its calibration run.

What is recorded. For each model call: tier, latency and tokens. For each audit and each path: compile time, start-up, and the result, wall-clock time and CPU time of each test class. For each model call the record also holds the reasoning tokens and the input tokens that the API reports as read from or written to its prompt cache. Costs are reported in time, CPU time, tokens and money, using one fixed price list [46]: \$0.10 and \$0.50 per million input and output tokens for Luna, \$2 and \$10 for Sol, and \$0.05 per CPU-hour of validation. A local model has no token price. It is charged in time, and in a sensitivity analysis at a stated price per GPU-hour.

Prompt caching. The API caches the prefix of any prompt of at least 1,024 tokens by default. A token read from the cache costs a tenth of the input price, and a token written to it 1.25 times [47]. What a request finds in the cache depends on what was sent in the minutes before it, so the bill of a recorded call is not a property of its session, and a replayed policy cannot be priced from it. Money is therefore reported in two readings that depend on the session alone. In the first, every input token pays the input price. In the second, a cache holds, within one session, every prefix that a tier has already sent: a blind call re-reads its prompt, and a round of a conversation reads

the previous prompt and writes what was added. The first reading ignores the cache; the second gives a session the most it can get from the cache on its own. The cache is expected to favour the strong tier, whose conversation sends its history again in every round. The conclusions of RQ3 and RQ5 are drawn from whichever reading shows the smaller saving, and the tokens that the API reported as cached are shown next to the model.

Reliability checks. Each test class runs three times on the fixed program; classes that change outcome are excluded as flaky. On a random tenth of the candidates, the audit is repeated in two other orders, to check that results do not depend on test order. Every candidate that passes the scheduled classes is run once on the whole suite: a failure there would mean that the skipped classes hid it, so the candidate is counted as rejected and the case is reported. That run is the CI run any proposed patch receives; it is the same under every policy and is not charged to the session.

5.4 E0 and E1: Single-Tier Sessions, Audited

The loops. The cheap tier draws candidates by blind re-sampling: each call sends the first prompt and returns $n = 5$ candidates, at temperature 1.0 and with reasoning off. The strong tier holds a conversation in the style of ChatRepair [3], one candidate per round, with the feedback of Table 5.3, at its default reasoning effort and sampling. The first prompt is the same for both tiers and gives the faulty methods, a failing test and its error message (Figure 5.1). A session ends at the first plausible patch or when its budget is spent. If an endpoint cannot return several candidates for one request, the five are drawn by five requests; the prompt is then paid five times, and the break-even is reported at that price.

Table 5.3: Feedback to the strong tier after each round.

Outcome	Feedback to the model
A test fails	the first failing test, its line and error; ask for a different fix
Does not compile	the compiler error; ask for a corrected version
Unreadable answer	the answer could not be read as the given methods
Repeat of its own candidate	the patch was already tried, and which test rejected it

Same draws for every policy. Model answers are cached under the full prompt and a draw index (bug, session, tier, call). The index omits the policy, so two policies that send the same prompt at the same point receive the same answer. Audits are cached by normalised patch, so each patch is tested once.

What E1 answers. The recorded sessions give RQ1: where the cost of each model goes, split into model calls, building, start-up and tests, and by how the session ended. Each session is priced twice: on the warm substrate, and as a plain loop that builds

```
The following Java method contains a bug:

    private static void modify(Calendar val, int field, boolean round) {
        ...
    }

It makes this test fail:
    Test:  DateUtilsTest::testTruncateLang59
    Error: AssertionFailedError: Truncate Calendar.SECOND
           expected:<Sun Oct 31 01:02:03 MDT 2004> but was:<... MST 2004>

Return the complete fixed method in one Java code block.
```

Figure 5.1: The first prompt for Defects4J bug Lang-65.

and starts a runtime for every candidate and validates repeats again. The sessions are also the streams that E2 replays.

What E0 answers. The audits of the same candidates give RQ2: what each step of a validation costs on each path, how often the two paths disagree on a verdict, how many bugs fail the calibration and fall back to the cold path, and how often a candidate forces a new runtime. They also give what the schedule leaves to any other order: the test time it spends before the first failure, against the cheapest rejecting test in hindsight.

5.5 E2: Escalation Policies by Replay

Why replay is exact. A policy with a cold handover never changes a prompt. What a model is sent depends only on its own earlier candidates and on what is reported about them. The reported failure of a candidate does not depend on the session (Section 4.3), and a repeat of the other tier's candidate is reported as an ordinary failure (Section 4.4). The session of such a policy is therefore a prefix of the recorded cheap session, followed by a prefix of the recorded strong session of the same bug and session number. Its cost is read from the records: the calls it would have made, and from the audits the validations it would have run, in the order its own memory would have scheduled them. Every policy thus sees exactly the same candidates, and the costs differ only by the policy. In the implementation a replay is the bot itself, run on recorded answers only: a policy that needed an answer nobody recorded stops with an error instead of calling a model.

What is measured. For each policy: the cost per repaired bug in money, tokens, time and CPU time; the share of sessions repaired at the cheap tier; the break-even margin of Proposition 2.3; the time spent at the cheap tier in sessions that step up; and the distance to Hindsight.

A check, not a test. In the main reading a policy repairs every session that Strong-only repairs. A session in which it does not is a defect of the harness, and is investigated as one.

5.6 E3 and E4: Handover and Whole System, Live

E3: handover. On 50 evaluation bugs, three sessions each, E2 says which sessions step up under Step. In each of them the strong tier is started again with each of three handovers: Evidence, Evidence and patches, and Transcript. The Cold handover is the recorded strong session itself. The four are compared on the same sessions: rounds of the strong tier until a plausible patch, repairs within its budget, and tokens. Two more rates are read from the Evidence Memory. The *refuted-again rate* is the share of strong candidates that fail with a failure signature the cheap tier had already produced. The *anchoring rate* is the share of strong candidates that equal a cheap candidate after normalising. A note that helps should lower the first without raising the second.

E4: whole system. The full bot runs on the other 50 evaluation bugs, three sessions each: Step, the handover chosen on E3 by the rule of Table 5.6, and the Budget Guard, which only logs when it would stop. Its stopping is then replayed, which separates the effect of stopping from the rest. The bot is compared with Strong-only on the same bugs, priced in two ways: with the same validation layer, which isolates escalation, handover and stopping; and as a plain conversational loop, which builds and starts a runtime for every candidate and validates repeats again. The second is the comparison of RQ5. The main measure is the cost per repaired bug; the repair rate must not fall by more than a margin fixed in advance. Choosing the handover on one half and testing the system on the other keeps the choice from being tested on the bugs that made it.

5.7 Metrics

Table 5.4 lists the metrics. Costs are summed over sessions before dividing, and every comparison is made per bug.

5.8 Statistics

The bug is the unit of analysis; the sessions of a bug are averaged first. The two main comparisons are Step against Strong-only in cost, and each note against Cold in strong rounds. They use a paired Wilcoxon test on the per-bug log-ratio, with a Holm correction, and are reported with bootstrap intervals over bugs. With 100 bugs, a saving of 5 % is detected with power 0.8 if the standard deviation of the log-ratio is at most 0.15. If the spread is larger, the smallest detectable saving is reported. The comparison of the notes has fewer units: only the bugs of E3 on which a session steps up, at most 50. Their number and the smallest difference they can detect are reported with the result, and a null there is read as an absence of evidence, not as equivalence.

In the main reading of E2 the repair rate cannot fall, so it is checked rather than tested (Section 5.5). In the equal-attempts reading, and in E3 and E4, the repair rate

Table 5.4: Metrics.

Metric	Definition	RQ
Cost per repaired bug	total cost of all sessions divided by the number of bugs repaired; in money (both readings of Section 5.3), tokens, time and CPU time	1, 3, 5
Cost breakdown	share of each tier's model calls, of building, of start-up and of tests in the total, by how the session ended	1
Validation cost per candidate	on the cold path and on M0; rate of disagreement in verdict; share of bugs that fall back	2
Cheap-tier repair share	share of sessions repaired before any strong call	3
Break-even margin	the cheap tier's repair share under the policy, minus its mean number of candidates times the price ratio c_1/c_2	3
Escalation delay	time spent at the cheap tier in sessions that step up	3
Distance to Hindsight	cost above Hindsight, as a share of the gap between Strong-only and Hindsight	3
Repair rate difference	share of sessions with a plausible patch, against Strong-only	3, 4, 5
Strong rounds to repair	rounds of the strong tier until a plausible patch	4
Refuted-again rate	share of strong candidates that fail with a signature the cheap tier had already produced	4
Anchoring rate	share of strong candidates equal, after normalising, to a cheap candidate	4
Cost of failed sessions	share of the total spent on sessions without a plausible patch	1, 5
Match with the developer's fix	share of plausible patches equal, after normalising, to the developer's fix, per tier	5

is tested for non-inferiority with a margin of 10 percentage points. With 50 bugs, that margin is reachable if the standard deviation of the paired difference is at most 0.24. If the development set shows more spread than assumed, the number of sessions per bug is raised before the number of bugs. The assumed values are provisional and checked on the development set (Table 5.6).

5.9 Compute and API Budget

Most of the compute goes to audits, and most of the money to the strong tier. Table 5.5 gives the counts. They are upper bounds: a session stops at the first plausible patch, and policies share cached answers.

The strong tier makes at most 9,000 calls. If every one carried 6,000 input and 3,000 output tokens, reasoning included, they would cost about $380 at list prices, and less with the prompt cache; the 1,200 cheap calls cost under $2. The development set adds at most 900 strong calls (20 bugs, three sessions, the 15 rounds of Table 5.6), about $40, and its cheap-tier variants a few dollars. If the estimate from the development set exceeds the budget, cuts are made in a fixed order: the Transcript handover of E3 is run on half of its bugs, then the sessions per bug of E3 are lowered from three to two. If the pair moves down one step, the bound falls by a factor of twenty.

Table 5.5: Calls and audits, as upper bounds.

Work	Upper bound
Flakiness check	3 suite runs per bug
E0	each distinct candidate audited on both paths
E1, cheap tier	100 bugs × 3 sessions × 4 calls of 5 candidates = 1,200 calls
E1, strong tier	100 bugs × 3 sessions × 10 rounds = 3,000 calls
E2	no model call
E3	50 bugs × 3 sessions × 3 notes × 10 rounds = 4,500 strong calls
E4	50 bugs × 3 sessions × 10 rounds = 1,500 strong calls; cheap calls shared with E1
Audits	one per distinct candidate, shared by all configurations; at most 4 CPU-hours per bug

5.10 Pre-Registration

This proposal fixes the bugs, settings, metrics and analyses in advance. Table 5.6 gives the values that still depend on data, each with a provisional value and the rule that fixes it. They are frozen before E1 starts on the evaluation set, and any later change is reported. The experiments run in order: E0 and E1, then E2, then E3, then E4.

Table 5.6: Provisional values and the rules that fix them.

Value	Provisional	Rule
Model pair	Luna, then Sol	development set: kept if the cheap tier repairs 20 % to 80 % of the bugs within its budget and the strong tier needs more than one round on at least a third of the rest; otherwise as in Section 5.1
Candidates per cheap call, n	5, at temperature 1.0	development set: the largest n of 1, 3, 5 and 10 at which at least half of the candidates of a call are distinct after normalising
Reasoning effort	cheap tier: none; strong tier: medium	development set: none is kept if the cheap tier still repairs at least 20 % of the bugs within its budget; otherwise low. The strong tier is not changed
Cheap budget B_1	20 candidates, for Step and for Cheap-only	not changed
Stall rule of the cheap tier	a call in which at most one candidate makes progress	development set: kept if it loses at most 5 % of the cheap tier's repairs; otherwise a call in which no candidate makes progress
Strong budget R_2	10 rounds	development set, run with 15 rounds: kept if fewer than 5 % of the repaired strong sessions need more than 10; otherwise 15
Stall count r of the strong tier	3 rounds without progress	development set: the smallest r whose replay loses at most 5 % of the plausible sessions
Session budget B	900 s of wall-clock time	development set: the 90th percentile of the time to a plausible patch in the Strong-only sessions
Handover used in E4	chosen on E3	on the 50 bugs of E3: the fewest strong rounds per repaired bug, with a repair rate within the margin of Cold; otherwise Cold
SD of the per-bug log-ratio	0.15	measured; if larger, the smallest detectable saving is reported
SD of the repair-rate difference	0.24	measured; if larger, the sessions per bug are raised, not the margin
Non-inferiority margin	10 percentage points	not changed
Bugs	120 accepted: 20 development, 100 evaluation (50 for E3, 50 for E4)	the share of Defects4J bugs that pass the rules is reported from the selection log

Chapter 6

Expected Outcomes

Each research question can come out in more than one way. Table 6.1 fixes in advance how each outcome will be read, so that negative results can be read as clearly as positive ones.

Two results hold whatever the data show. The cost breakdown of RQ1 is a result in any case. And under a cold handover with an unchanged strong budget, the repair rate cannot fall (Proposition 2.2). The open question of RQ3 is therefore the size and sign of the saving, not a loss of repairs.

The expected picture is a saving of tens of percent in money and tokens, a small change in time, and an open answer on the handover. Escalation is expected to help most where the price gap is wide and the cheap tier repairs a fair share of the bugs (Section 4.7). The thesis reports which of these conditions hold on the benchmark rather than assuming them.

Table 6.1: Possible outcomes and how they will be read.

RQ	If ...	then ...
RQ1	the strong tier's calls dominate the cost	escalation is the main lever; its saving is bounded by the strong tier's spend on bugs the cheap tier repairs
RQ1	validation dominates, even on the warm substrate	the tests are the cost; selection and the schedule matter more than the choice of model, and the saving of escalation is reported as diluted
RQ1	most of the spend falls on bugs that are never repaired	no choice of model removes it; the Budget Guard is the lever, and its trade-off is reported
RQ2	M0 removes most of the build and start-up	cheap candidates cost little to check, and cheap-first costs little time
RQ2	M0 disagrees with cold validation on some candidates	the rate and its causes are reported; the cold path is used wherever the verdict cannot be guaranteed
RQ2	many bugs fail the calibration, or many candidates force a new runtime	the fallback rate bounds the saving, and those validations are charged at the cold price or with the new start
RQ2	the schedule leaves much to a better order	the room is reported as the bound on what a new ordering could add; it is not pursued here
RQ3	Step costs less than Strong-only	the bot meets the need; the breakdown shows where the saving came from

RQ	If ...	then ...
RQ3	the cheap tier repairs too few bugs to pay	the break-even margin is negative and says by how much; for this setting the recommendation is to start at the strong tier
RQ3	Step is no better than Fixed-5 or Fixed-10	the simpler rule is recommended; the stall signal adds nothing here
RQ3	Step is close to Hindsight	little is left for any router in test-driven repair, learned ones included
RQ3	Step is far from Hindsight	the gap is the room for a better trigger; whether it comes from hopeless bugs kept too long or from easy bugs given up too early is reported
RQ3	money falls but time rises	the two prices are reported separately; which one matters is a property of the deployment
RQ3	with equal attempts, the repair rate falls	the cheap attempts were worth less than the strong rounds they replaced; the budget split at which the loss disappears is reported
RQ4	a note lowers the strong tier's rounds	verified evidence transfers; the note is worth its tokens and becomes the default
RQ4	no note differs from Cold	evidence does not transfer for this pair; cold start is recommended, in agreement with the fresh start of SWE-Router [16]
RQ4	a note raises the anchoring rate or lowers repairs	the bias reported for drafts [17] extends to this handover; the variant without patches says whether the drafts or the facts caused it
RQ4	Transcript does as well as the notes	the summary loses nothing, and the shorter note is preferred for its tokens
RQ4	the strong tier repairs nearly every bug in its first round	there is no room to measure a handover; RQ4 is reported on the pair one step down
RQ5	the bot costs less at the same repair rate	the system meets the need on this benchmark
RQ5	the saving comes from the validation layer, not from escalation	the two prices of Strong-only separate them; the layer is recommended on its own, and escalation only where its break-even holds
RQ5	no saving end to end	the saving is diluted by costs the system does not touch, mainly the strong tier's spend on bugs nobody repairs; the breakdown says which
RQ5	cheap-tier patches match the developer's fix less often	plausible is not correct, and the cheap tier may overfit more; the saving is reported next to this gap

Chapter 7

Work Plan and Conclusion

7.1 Work Plan

The work takes four months (Table 7.1). Everything before the registration point prepares the experiments; everything after it runs them as planned.

Table 7.1: Work plan.

Month	Work	Milestone
1	The warm substrate and its calibration; the two-tier loop with shared caches; the Evidence Memory across tiers; Step and the handover note; development set: model pair, variants, compute estimate, all settings fixed	a tiered session runs on every development bug
2	E0 and E1: at most 1,200 cheap and 3,000 strong calls, every candidate audited on both paths; E2 by replay	**registration**; RQ1 to RQ3 analysed
3	E3: handovers on 50 bugs; E4 on the other 50	RQ4 and RQ5 analysed
4	Writing; release of the bot, as a command-line tool with a CI workflow, and of the data	thesis draft

7.2 Risks

Table 7.2 lists the main risks and how each is handled.

Table 7.2: Main risks and how they are handled.

Risk	Handling
Escalation saves little	the cost breakdown (C1) and the break-even stand on their own; the result is read as in Chapter 6
Step only matches a fixed switch	reported as a finding, with the simpler rule recommended
The handover shows no effect	cold start is the default and keeps its guarantee; a null answers an open question
The benchmark is memorised, so the strong tier repairs everything at once	the model pair is chosen on the development set by a fixed rule, and can move down one step
The warm path disagrees with ordinary validation	every candidate is also validated on the cold path, whose verdict is the one of record; a bug that fails the calibration falls back
The model service changes	versions pinned, answers cached; E2 needs no model
The API bill is too high	the upper bound is known in advance, with a fixed order of cuts (Section 5.9)
Audits take too long	caps per bug
Flaky or order-dependent tests	detected by repeated runs and excluded
Four months is tight	the Transcript handover is cut first, then E4; E0 to E2 answer RQ1 to RQ3

7.3 Threats to Validity

Internal. Replay assumes that test results and times do not depend on order, and that a cold strong session does not depend on the cheap tier. The first is checked on a sample of audits. The second holds by construction, since what a model is told about a candidate is fixed by the candidate alone; it is also checked, because the replay runs the bot itself and fails if a session asks for an answer that was not recorded. Class times are measured once per candidate and path, so a charge does not capture how the warmth of the runtime changes within a session. Model answers are cached, so every comparison is conditional on one table of draws; the replication is over bugs and sessions, not over re-sampled answers.

External. One benchmark, one language, one model family and two tiers; the faulty methods are given to the model. Defects4J may be memorised [27], which inflates the cheap tier's repair share and with it the saving. Each tier runs at one reasoning effort. Prices and the terms of the prompt cache change, so the break-even is also reported as a ratio that can be re-read at other prices. CI systems that run tests in parallel on many machines are not covered, and neither are bugs without a failing test.

Construct. Success means a plausible patch, not a correct one. A cheap model may produce plausible but wrong patches more often than a strong one, and the bot returns the first plausible patch it finds. The share of plausible patches that match the

developer's fix is reported per tier as a partial check; judging correctness in general is out of scope.

7.4 Future Work

A benchmark newer than the models. The clearest extension is a set of bugs that the models cannot have seen. It removes the memorisation that inflates the cheap tier's repair share on Defects4J, and a second benchmark would show whether the break-even moves as its three inputs predict.

More tiers and local models. A local model has no token price but costs time and energy, and smaller is not always cheaper [37]. A third tier would test whether the break-even composes.

A learned trigger on top of the verified one. If Step ends far from Hindsight, the gap is the room for a predictor such as those of Section 3.5, used only where the tests leave the decision open.

Other settings. Python, using BugsInPy [48], where there is no build step and one validation is cheaper; and repository-level issue resolution [49], where a failing test must first be written before a stall can be verified.

Correctness across tiers. Whether patches from the cheap tier are correct as often as those from the strong tier, judged by developers or by held-out tests.

7.5 Conclusion

In LLM-based repair, each round costs a model call and a test run. Known techniques make the test run cheap. This proposal turns to the model call. It builds a tiered repair bot that tries a cheap model first and steps up to a strong model only when the tests show a stall. A Warm Substrate makes cheap candidates cheap to check. An Evidence Memory keeps what the tests have shown across the switch. An Escalation Controller applies the rule Step and writes a handover note. A Budget Guard stops the session and explains why. With a cold handover the bot loses no repair that the strong model alone would make, so what the evaluation measures is its cost: by replay on recorded sessions against a hindsight bound, and in live runs on Defects4J, with the reading of every outcome fixed in advance.

References

[1] Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. Automated program repair. *Communications of the ACM*, 62(12):56–65, 2019. doi:10.1145/3318162.

[2] Zichao Qi, Fan Long, Sara Achour, and Martin Rinard. An analysis of patch plausibility and correctness for generate-and-validate patch generation systems. In *ISSTA 2015*, pages 24–36. doi:10.1145/2771783.2771791.

[3] Chunqiu Steven Xia and Lingming Zhang. Automated program repair via conversation: Fixing 162 out of 337 bugs for $0.42 each using ChatGPT. In *ISSTA 2024*, pages 819–831. doi:10.1145/3650212.3680323.

[4] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. RepairAgent: An autonomous, LLM-based agent for program repair. In *ICSE 2025*, pages 2188–2200. doi:10.1109/ICSE55347.2025.00157.

[5] Boyang Yang, Zijian Cai, Fengling Liu, Bach Le, Lingming Zhang, Tegawendé F. Bissyandé, Yang Liu, and Haoye Tian. A survey of LLM-based software repair: Taxonomies, design paradigms, and applications. arXiv:2506.23749v2, 2025.

[6] Yannic Noller, Ridwan Shariffdeen, Xiang Gao, and Abhik Roychoudhury. Trust enhancement issues in program repair. In *ICSE 2022*, pages 2228–2240. doi:10.1145/3510003.3510040.

[7] AI Business. OpenAI takes AI price war to next level with GPT-6 Sol, Luna pricing. 23 September 2026. aibusiness.com/generative-ai/openai-takes-ai-price-war-next-level-gpt-6-sol-luna-pricing.

[8] Lingchao Chen, Yicheng Ouyang, and Lingming Zhang. Fast and precise on-the-fly patch validation for all. In *ICSE 2021*, pages 1123–1134. doi:10.1109/ICSE43902.2021.00104.

[9] Yuan-An Xiao, Chenyang Yang, Bo Wang, and Yingfei Xiong. Accelerating patch validation for program repair with interception-based execution scheduling. *IEEE Transactions on Software Engineering*, 50(3):618–635, 2024. doi:10.1109/TSE.2024.3359969.

[10] Yiling Lou, Jun Yang, Samuel Benton, Dan Hao, Lin Tan, Zhenpeng Chen, Lu Zhang, and Lingming Zhang. When automated program repair meets regression testing: An extensive study on two million patches. *ACM Transactions on Software Engineering and Methodology*, 33(7):180:1–180:23, 2024. doi:10.1145/3672450.

[11] Yuhua Qi, Xiaoguang Mao, and Yan Lei. Efficient automated program repair through fault-recorded testing prioritization. In *ICSM 2013*, pages 180–189. doi:10.1109/ICSM.2013.29.

[12] Henry Coles, Thomas Laurent, Christopher Henard, Mike Papadakis, and Anthony Ventresque. PIT: A practical mutation testing tool for Java (demo). In *ISSTA 2016*, pages 449–452. doi:10.1145/2931037.2948707.

[13] Aolin Chen, Haojun Wu, Qi Xin, Steven P. Reiss, and Jifeng Xuan. Studying and understanding the effectiveness and failures of conversational LLM-based repair. In *2025 IEEE/ACM International Workshop on Automated Program Repair (APR)*, pages 56–59. doi:10.1109/APR66717.2025.00014.

[14] Yuvraj Verma. Try again, don't look back: Blind resampling outperforms self-repair in small code models. arXiv:2607.26117, 2026.

[15] Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. Is self-repair a silver bullet for code generation? In *ICLR 2024*.

[16] Seongho Son, Sangwoong Yoon, Jiahua Tang, Shuhan Wang, Lorenz Wolf, and Ilija Bogunovic. SWE-Router: Routing in multi-turn agentic software engineering tasks. arXiv:2607.00053, 2026. Poster at the ICML 2026 workshop Deep Learning for Code.

[17] Jingjie Ning, Xueqi Li, and Chengyu Yu. Revision or re-solving? Decomposing second-pass gains in multi-LLM pipelines. arXiv:2604.01029, 2026.

[18] Chenyu Wang, Yunbo Lyu, Junda He, Zhou Yang, Chenxing Zhong, Yaniv Harel, and David Lo. Fail-fast, restart-smart: Early failure prediction and restart for SWE agentic tasks. arXiv:2608.03222, 2026.

[19] Junchi Liu, Ali Bigdeli, Roya Daneshi, Atu Ambala, Sudipto Ghosh, and Fabio Santos. Rethinking automated program repair: The impact of bug complexity, fault localization, and LLM cost-efficiency. arXiv:2608.14065, 2026.

[20] Jiajun Sun, Fengjie Li, Xinzhu Qi, Hongyu Zhang, and Jiajun Jiang. Empirical evaluation of large language models in automated program repair. arXiv:2506.13186, 2025.

[21] Zhiyu Fan, Kirill Vasilevski, Dayi Lin, Boyuan Chen, Yihao Chen, Zhiqing Zhong, Jie M. Zhang, Pinjia He, and Ahmed E. Hassan. SWE-Effi: Re-evaluating software AI agent system effectiveness under resource constraints. arXiv:2509.09853, 2025.

[22] Boyuan Chen, Mingzhi Zhu, Brendan Dolan-Gavitt, Muhammad Shafique, and Siddharth Garg. Model cascading for code: A cascaded black-box multi-model

framework for cost-efficient code completion with self-testing. arXiv:2405.15842, 2024.

[23] Shubham Gandhi, Atharva Naik, Yiqing Xie, and Carolyn Rose. An empirical study on strong-weak model collaboration for repo-level code generation. arXiv:2505.20182, 2025.

[24] Yaoqi Guo, Ying Xiao, Jie M. Zhang, Mark Harman, Yiling Lou, Yang Liu, and Zhenpeng Chen. EET: Experience-driven early termination for cost-efficient software engineering agents. arXiv:2601.05777, 2026.

[25] René Just, Darioush Jalali, and Michael D. Ernst. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *ISSTA 2014*, pages 437–440. doi:10.1145/2610384.2628055.

[26] Lech Madeyski. Triage: Routing software engineering tasks to cost-effective LLM tiers via code quality signals. arXiv:2604.07494, 2026.

[27] Daniel Ramos, Claudia Mamede, Kush Jain, Paulo Canelas, Catarina Gamboa, and Claire Le Goues. Are large language models memorizing bug benchmarks? arXiv:2411.13323, 2024.

[28] Zhihao Lin, Junhua Zhu, Mingyi Zhou, Xin Wang, Zhensu Sun, Renyu Yang, David Lo, and Li Li. To run or not to run: Analyzing the cost-effectiveness of code execution in LLM-based program repair. *Proceedings of the ACM on Software Engineering*, 3(ISSTA):503–525, 2026. doi:10.1145/3832113.

[29] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. arXiv:2305.05176, 2023.

[30] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *NeurIPS 2024*.

[31] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. In *ISSTA 2024*, pages 1592–1604. doi:10.1145/3650212.3680384.

[32] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Demystifying LLM-based software engineering agents. *Proceedings of the ACM on Software Engineering*, 2(FSE):801–824, 2025. doi:10.1145/3715754.

[33] Alexandru Marginean, Johannes Bader, Satish Chandra, Mark Harman, Yue Jia, Ke Mao, Alexander Mols, and Andrew Scott. SapFix: Automated end-to-end repair at scale. In *ICSE-SEIP 2019*, pages 269–278. doi:10.1109/ICSE-SEIP.2019.00039.

[34] Chandra Maddila et al. Agentic program repair from test failures at scale. arXiv:2507.18755, 2025.

[35] Pat Rondon, Renyao Wei, José Cambronero, Jürgen Cito, Aaron Sun, Siddhant Sanyam, Michele Tufano, and Satish Chandra. Evaluating agent-based program repair at Google. In *ICSE-SEIP 2025*, pages 365–376. doi:10.1109/ICSE-SEIP66354.2025.00038.

[36] Dávid Hidvégi, Khashayar Etemadi, Sofia Bobadilla, and Martin Monperrus. CigaR: Cost-efficient program repair with LLMs. arXiv:2402.06598, 2024.

[37] Fernando Vallecillos-Ruiz, Giordano d'Aloisio, Max Hort, Luca Traini, Antinisca Di Marco, and Leon Moonen. Smaller models, unexpected costs: Trade-offs in LLM quantization for automated program repair. arXiv:2606.27205, 2026.

[38] Sergey Mechtaev, Xiang Gao, Shin Hwei Tan, and Abhik Roychoudhury. Test-equivalence analysis for automatic patch generation. *ACM Transactions on Software Engineering and Methodology*, 27(4):15:1–15:37, 2018. doi:10.1145/3241980.

[39] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. arXiv:2406.18665, 2024.

[40] Hao Tang, Keya Hu, Jin Peng Zhou, Sicheng Zhong, Wei-Long Zheng, Xujie Si, and Kevin Ellis. Code repair with LLMs gives an exploration-exploitation tradeoff. In *NeurIPS 2024*. arXiv:2405.17503.

[41] Kai Ruan, Zihe Huang, Ziqi Zhou, Qianshan Wei, Jinghao Lin, Xuan Wang, and Hao Sun. Doomed from the start: Early abort of LLM agent episodes via a recall-controlled probe cascade. arXiv:2607.06503, 2026.

[42] Serkan Kirbas, Etienne Windels, Olayori McBello, Kevin Kells, Matthew Pagano, Rafal Szalanski, Vesna Nowack, Emily Rowan Winter, Steve Counsell, David Bowes, Tracy Hall, Sæmundur Haraldsson, and John Woodward. On the introduction of automatic program repair in Bloomberg. *IEEE Software*, 38(4):43–51, 2021. doi:10.1109/MS.2021.3071086.

[43] Emily Winter, David Bowes, Steve Counsell, Tracy Hall, Sæmundur Haraldsson, Vesna Nowack, and John Woodward. How do developers really feel about bug fixing? Directions for automatic program repair. *IEEE Transactions on Software Engineering*, 49(4):1823–1841, 2023. doi:10.1109/TSE.2022.3194188.

[44] Emily Rowan Winter, Vesna Nowack, David Bowes, Steve Counsell, Tracy Hall, Sæmundur Haraldsson, John Woodward, Serkan Kirbas, Etienne Windels, Olayori McBello, Abdurahman Atakishiyev, Kevin Kells, and Matthew Pagano. Towards developer-centered automatic program repair: Findings from Bloomberg. In *ESEC/FSE 2022*, pages 1578–1588. doi:10.1145/3540250.3558953.

[45] Kui Liu, Anil Koyuncu, Tegawendé F. Bissyandé, Dongsun Kim, Jacques Klein, and Yves Le Traon. You cannot fix what you cannot find! An investigation of fault localization bias in benchmarking automated program repair systems. In *ICST 2019*, pages 102–113. doi:10.1109/ICST.2019.00020.

[46] OpenAI. GPT-6 Luna and GPT-6 Sol: Model reference. developers.openai.com/api/docs/models/gpt-6-luna, developers.openai.com/api/docs/models/gpt-6-sol, read November 2026.

[47] OpenAI. Prompt caching. developers.openai.com/api/docs/guides/prompt-caching, read November 2026.

[48] Ratnadira Widyasari et al. BugsInPy: A database of existing bugs in Python programs to enable controlled testing and debugging studies. In *ESEC/FSE 2020*, pages 1556–1560. doi:10.1145/3368089.3417943.

[49] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR 2024*.